

Operating System

CPU Scheduling Algorithms [FCFS, SJF, RR, Priority]

FCFS

```
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int AT[n], BT[n], CT[n], TAT[n], WT[n];

    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
        scanf("%d %d", &AT[i], &BT[i]);
    }

    // Completion Time calculation
    CT[0] = AT[0] + BT[0];

    for(i = 1; i < n; i++) {
        if(CT[i-1] < AT[i])
            CT[i] = AT[i] + BT[i];
        else
            CT[i] = CT[i-1] + BT[i];
    }

    float totalTAT = 0, totalWT = 0;

    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];

        totalTAT += TAT[i];
        totalWT += WT[i];
    }

    printf("\nAverage Turnaround Time: %.2f\n", totalTAT/n);
    printf("Average Waiting Time: %.2f\n", totalWT/n);

    return 0;
}
```

SJF

```
#include <stdio.h>

int main() {
    int n, i, j, time = 0, completed = 0, min;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    int AT[n], BT[n], CT[n], TAT[n], WT[n], visited[n];
    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
        scanf("%d %d", &AT[i], &BT[i]);
        visited[i] = 0;
    }
    while(completed < n) {
        min = -1;

        for(i = 0; i < n; i++) {
            if(AT[i] <= time && visited[i] == 0) {
                if(min == -1 || BT[i] < BT[min]) {
                    min = i;
                }
            }
        }

        if(min == -1) {
            time++;
        }
        else {
            time += BT[min];
            CT[min] = time;
            visited[min] = 1;
            completed++;
        }
    }

    float totalTAT = 0, totalWT = 0;
    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];

        totalTAT += TAT[i];
        totalWT += WT[i];
    }
    printf("\nAverage Turnaround Time = %.2f\n", totalTAT/n);
    printf("Average Waiting Time = %.2f\n", totalWT/n);
    return 0;
}
```

PRIORITY

```
#include <stdio.h>
int main() {
    int n, i, time = 0, completed = 0, min;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int AT[n], BT[n], P[n], CT[n], TAT[n], WT[n], visited[n];
    for(i = 0; i < n; i++) {
        printf("Enter AT, BT and Priority for P%d: \n", i+1);
        scanf("%d %d %d", &AT[i], &BT[i], &P[i]);
        visited[i] = 0;
    }
    while(completed < n) {
        min = -1;
        for(i = 0; i < n; i++) {
            if(AT[i] <= time && visited[i] == 0) {
                if(min == -1 || P[i] < P[min]) {
                    min = i;
                }
            }
        }
        if(min == -1) {
            time++;
        }
        else {
            time += BT[min];
            CT[min] = time;
            visited[min] = 1;
            completed++;
        }
    }

    float totalTAT = 0, totalWT = 0;
    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];

        totalTAT += TAT[i];
        totalWT += WT[i];
    }
    printf("\nAverage Turnaround Time = %.2f\n", totalTAT/n);
    printf("Average Waiting Time = %.2f\n", totalWT/n);
    return 0;
}
```

ROUND ROBIN

```
#include <stdio.h>

int main() {
    int n, tq, i, time = 0, completed = 0;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    printf("Enter Time Quantum: \n");
    scanf("%d", &tq);

    int AT[n], BT[n], RT[n], CT[n], TAT[n], WT[n];

    for(i = 0; i < n; i++) {
        printf("Enter AT and BT for P%d: \n", i+1);
        scanf("%d %d", &AT[i], &BT[i]);
        RT[i] = BT[i];
    }

    while(completed < n) {
        int done = 1;

        for(i = 0; i < n; i++) {
            if(RT[i] > 0 && AT[i] <= time) {
                done = 0;
                if(RT[i] > tq) {
                    time += tq;
                    RT[i] -= tq;
                }
                else {
                    time += RT[i];
                    CT[i] = time;
                    RT[i] = 0;
                    completed++;
                }
            }
        }

        if(done)
            time++;
    }

    float totalTAT = 0, totalWT = 0;
    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];
    }
}
```

```

        totalTAT += TAT[i];
        totalWT += WT[i];
    }

    printf("\nAverage Turnaround Time: %.2f\n", totalTAT/n);
    printf("Average Waiting Time: %.2f\n", totalWT/n);

    return 0;
}

```

BANKERS ALGO

```

#include <stdio.h>

int main() {
    int p, r, i, j, k;

    printf("Enter number of processes: \n");
    scanf("%d", &p);

    printf("Enter number of resource types: \n");
    scanf("%d", &r);

    int alloc[p][r], max[p][r], need[p][r];
    int avail[r], work[r];
    int finish[p], safeSeq[p];

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < p; i++)
        for(j = 0; j < r; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for(i = 0; i < p; i++)
        for(j = 0; j < r; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available Resources: \n");
    for(i = 0; i < r; i++) {
        scanf("%d", &avail[i]);
        work[i] = avail[i];
    }

    for(i = 0; i < p; i++)
        for(j = 0; j < r; j++)
            need[i][j] = max[i][j] - alloc[i][j];
}

```

```

for(i = 0; i < p; i++)
    finish[i] = 0;

int count = 0;

while(count < p) {
    int found = 0;

    for(i = 0; i < p; i++) {
        if(finish[i] == 0) {
            int possible = 1;

            for(j = 0; j < r; j++) {
                if(need[i][j] > work[j]) {
                    possible = 0;
                    break;
                }
            }

            if(possible) {
                for(k = 0; k < r; k++)
                    work[k] += alloc[i][k];

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(!found)
        break;
}

if(count == p) {
    printf("System is in a SAFE STATE.\n");
    printf("Safe Sequence: ");
    for(i = 0; i < p; i++)
        printf("P%d ", safeSeq[i]);
}
else {
    printf("System is NOT in a safe state (Deadlock may occur).\n");
}
return 0;
}

```

Producer Consumer

```
#include <stdio.h>
#include <stdlib.h>

int mutex = 1, full = 0, empty = 5, x = 0;

int wait(int s)
{
    return --s;
}

int signal(int s)
{
    return ++s;
}

void producer()
{
    mutex = wait(mutex);

    empty = wait(empty);
    full = signal(full);

    x++;
    printf("\nProducer produces item %d", x);

    mutex = signal(mutex);
}

void consumer()
{
    mutex = wait(mutex);

    full = wait(full);
    empty = signal(empty);

    printf("\nConsumer consumes item %d", x);
    x--;

    mutex = signal(mutex);
}

int main()
{
    int n;
```

```

printf("\n----- Producer Consumer Problem -----");
printf("\n1. Producer");
printf("\n2. Consumer");
printf("\n3. Exit");

while(1)
{
    printf("\nEnter your choice: ");
    scanf("%d",&n);

    switch(n)
    {
        case 1:
            if(mutex==1 && empty!=0)
            {
                producer();
            }
            else
            {
                printf("\nBuffer is full!! Producer cannot produce.");
            }
            break;

        case 2:
            if(mutex==1 && full!=0)
            {
                consumer();
            }
            else
            {
                printf("\nBuffer is empty!! Consumer cannot consume.");
            }
            break;

        case 3:
            printf("\nExiting program...");
            printf("\nFinal buffer items produced: %d\n", x);
            exit(0);

        default:
            printf("\nInvalid choice. Try again.");
    }
}

return 0;
}

```

Memory Management Techniques [First Fit, Best Fit]

First Fit

```
#include <stdio.h>
```

```
int main() {
```

```
    int m, n;
```

```
    // =====
```

```
    // INPUT NUMBER OF MEMORY BLOCKS
```

```
    // =====
```

```
    printf("Enter number of memory blocks:\n");
```

```
    scanf("%d", &m);
```

```
    /*
```

```
    Example:
```

```
    m = 5 blocks
```

```
    Blocks:
```

```
    B1, B2, B3, B4, B5
```

```
    */
```

```
    // =====
```

```
    // DECLARING BLOCK ARRAYS
```

```
    // =====
```

```
    int blockSize[m];      // stores current (remaining) size of each block
```

```
    int originalBlockSize[m]; // stores original size for display purpose
```

```
    // =====
```

```
    // INPUT BLOCK SIZES
```

```
    // =====
```

```
    printf("Enter size of each block:\n");
```

```
    /*
```

```
    Example:
```

```
    Block Sizes:
```

```
    100 500 200 300 600
```

```
    */
```

```
    for(int i = 0; i < m; i++) {
```

```

scanf("%d", &blockSize[i]);

/*
Store original size separately
because blockSize[] will change
after allocation (splitting)
*/

originalBlockSize[i] = blockSize[i];
}

// =====
// INPUT NUMBER OF PROCESSES
// =====

printf("Enter number of processes:\n");
scanf("%d", &n);

/*
Example:

n = 4 processes

Processes:
P1, P2, P3, P4
*/

// =====
// DECLARING PROCESS ARRAY
// =====

int processSize[n];

// =====
// INPUT PROCESS SIZES
// =====

printf("Enter size of each process:\n");

/*
Example:

Process Sizes:
212 417 112 426
*/

for(int i = 0; i < n; i++) {
    scanf("%d", &processSize[i]);
}

```

```

}

// =====
// FIRST FIT ALGORITHM STARTS
// =====

/*
First Fit Strategy:

- For each process, scan blocks from beginning
- Allocate the FIRST block that is large enough
- Reduce the block size (memory splitting)
*/

for(int i = 0; i < n; i++) {

    int allocated = 0;

    /*
    allocated = 1 → process allocated
    allocated = 0 → process not allocated
    */

    // =====
    // SEARCH BLOCKS SEQUENTIALLY
    // =====

    for(int j = 0; j < m; j++) {

        /*
        Condition to allocate:
        block must have enough space
        */

        if(blockSize[j] >= processSize[i]) {

            /*
            Internal Fragmentation:
            leftover space in that block
            */

            int fragment = blockSize[j] - processSize[i];

            // =====
            // PRINT ALLOCATION DETAILS
            // =====

```

```

        printf("Process %d of size %d is allocated to Block %d of size %d with Fragment
%d\n",
               i + 1, processSize[i], j + 1,
               originalBlockSize[j], fragment);

        /*
        Reduce block size after allocation
        (remaining memory can be reused)
        */

        blockSize[j] -= processSize[i];

        allocated = 1;

        /*
        Break because First Fit uses
        the FIRST suitable block only
        */

        break;
    }
}

// =====
// IF NO BLOCK FOUND
// =====

if(!allocated) {

    printf("Process %d of size %d is not allocated\n",
           i + 1, processSize[i]);
}

return 0;
}

```

Best Fit

```

#include <stdio.h>

int main() {

    int m, n;

```

```

// =====
// INPUT NUMBER OF MEMORY BLOCKS
// =====

printf("Enter number of memory blocks:\n");
scanf("%d", &m);

/*
Example:

m = 5 blocks

Blocks:
B1, B2, B3, B4, B5
*/

// =====
// DECLARING BLOCK ARRAYS
// =====

int blockSize[m]; // stores remaining size of each block
int original[m];  // stores original size for display

// =====
// INPUT BLOCK SIZES
// =====

printf("Enter size of each block:\n");

/*
Example:

Block Sizes:
100 500 200 300 600
*/

for(int i = 0; i < m; i++) {
    scanf("%d", &blockSize[i]);

    /*
    Save original block size
    because blockSize[] will change
    after allocation
    */

    original[i] = blockSize[i];
}

```

```
// =====  
// INPUT NUMBER OF PROCESSES  
// =====
```

```
printf("Enter number of processes:\n");  
scanf("%d", &n);
```

```
/*
```

Example:

n = 4 processes

Processes:

P1, P2, P3, P4

```
*/
```

```
// =====  
// DECLARING PROCESS ARRAY  
// =====
```

```
int processSize[n];
```

```
// =====  
// INPUT PROCESS SIZES  
// =====
```

```
printf("Enter size of each process:\n");
```

```
/*
```

Example:

Process Sizes:

212 417 112 426

```
*/
```

```
for(int i = 0; i < n; i++) {  
    scanf("%d", &processSize[i]);  
}
```

```
// =====  
// BEST FIT ALGORITHM STARTS  
// =====
```

```
/*
```

Best Fit Strategy:

- For each process:
 - Check ALL memory blocks

→ Choose the block with
MINIMUM leftover space

- This reduces wastage
*/

```
for(int i = 0; i < n; i++) {
```

```
    int bestIndex = -1;
```

```
    /*
```

```
    bestIndex stores the index  
    of the best suitable block
```

```
    -1 means no block found yet  
    */
```

```
    // =====  
    // SEARCH ALL BLOCKS  
    // =====
```

```
    for(int j = 0; j < m; j++) {
```

```
        /*
```

```
        Check if block can fit process  
        */
```

```
        if(blockSize[j] >= processSize[i]) {
```

```
            /*
```

```
            Select block if:
```

```
            1. No block selected yet
```

```
            OR
```

```
            2. Current block is smaller  
               than previously selected block
```

```
            */
```

```
            if(bestIndex == -1 || blockSize[j] < blockSize[bestIndex]) {  
                bestIndex = j;
```

```
            }
```

```
        }
```

```
    }
```

```
    // =====  
    // IF BEST BLOCK FOUND  
    // =====
```

```
    if(bestIndex != -1) {
```

```

/*
Internal Fragmentation:
leftover space after allocation
*/

int fragment = blockSize[bestIndex] - processSize[i];

// =====
// PRINT ALLOCATION DETAILS
// =====

printf("Process %d of size %d is allocated to Block %d of size %d with Fragment
%d\n",
        i + 1, processSize[i], bestIndex + 1,
        original[bestIndex], fragment);

/*
Reduce block size after allocation
(remaining memory reused)
*/

blockSize[bestIndex] -= processSize[i];
}

// =====
// IF NO BLOCK FOUND
// =====

else {

    printf("Process %d of size %d is not allocated\n",
            i + 1, processSize[i]);
}
}

return 0;
}

```

Page Replacement Algorithms [FIFO, LRU]

FIFO

```
#include <stdio.h>

int main() {

    int n, frames;

    // =====
    // INPUT NUMBER OF PAGES
    // =====

    printf("Enter number of pages in reference string: \n");
    scanf("%d", &n);

    /*
    Example:

    n = 7 pages

    Reference String:
    1 3 0 3 5 6 3
    */

    // =====
    // DECLARE REFERENCE STRING ARRAY
    // =====

    int ref[n];

    // =====
    // INPUT REFERENCE STRING
    // =====

    printf("Enter the reference string: \n");

    for(int i = 0; i < n; i++) {
        scanf("%d", &ref[i]);
    }

    // =====
    // INPUT NUMBER OF FRAMES
    // =====

    printf("Enter number of frames: \n");
```

```

scanf("%d", &frames);

/*
Frames = Number of memory slots available
Example:

frames = 3
*/

// =====
// DECLARE FRAME ARRAY
// =====

int frame[frames];

// Initialize frames as empty (-1)
for(int i = 0; i < frames; i++) {
    frame[i] = -1;
}

// =====
// FIFO POINTERS AND COUNTERS
// =====

int front = 0; // points to the oldest page in frames
int faults = 0; // counts page faults
int hits = 0; // counts page hits

// =====
// FIFO PAGE REPLACEMENT LOOP
// =====

for(int i = 0; i < n; i++) {

    int found = 0; // flag to check if page is already in frame

    // =====
    // CHECK FOR PAGE HIT
    // =====

    for(int j = 0; j < frames; j++) {

        if(frame[j] == ref[i]) {

            /*
            Page is already in memory
            → HIT
            */

```

```

        found = 1;
        hits++;
        break;
    }
}

// =====
// PAGE FAULT HANDLING
// =====

if(!found) {

    /*
    Page is not in memory → FAULT

    FIFO Replacement:

    - Replace the oldest page (pointed by `front`)
    - Move `front` to next frame cyclically
    */

    frame[front] = ref[i];
    front = (front + 1) % frames;

    faults++;
}

}

// =====
// FINAL OUTPUT
// =====

printf("\nTotal Page Faults: %d\n", faults);
printf("Total Page Hits: %d\n", hits);

return 0;
}

```

//LRU

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, frames;
```

```

// =====
// INPUT NUMBER OF PAGES
// =====

printf("Enter number of pages in reference string: \n");
scanf("%d", &n);

/*
Example:

n = 5 pages

Reference String:
1 2 3 4 5
*/

// =====
// DECLARE REFERENCE STRING ARRAY
// =====

int ref[n];

// =====
// INPUT REFERENCE STRING
// =====

printf("Enter the reference string: \n");

for(int i = 0; i < n; i++) {
    scanf("%d", &ref[i]);
}

// =====
// INPUT NUMBER OF FRAMES
// =====

printf("Enter number of frames: \n");
scanf("%d", &frames);

/*
frames = number of memory slots available
Example: frames = 3
*/

// =====
// DECLARE FRAME AND TIME ARRAYS
// =====

```

```

int frame[frames]; // stores pages in memory
int time[frames]; // stores "last used time" for each frame

// =====
// INITIALIZATION
// =====

for(int i = 0; i < frames; i++) {
    frame[i] = -1; // empty frame
    time[i] = 0; // time = 0 initially
}

int faults = 0; // counts page faults
int hits = 0; // counts page hits
int counter = 0; // global counter to track page usage

// =====
// LRU PAGE REPLACEMENT LOOP
// =====

for(int i = 0; i < n; i++) {

    int found = 0; // flag to check if page is already in memory

    // =====
    // CHECK FOR PAGE HIT
    // =====

    for(int j = 0; j < frames; j++) {
        if(frame[j] == ref[i]) {

            /*
            Page is already in memory → HIT
            Update time of this frame to current counter
            */

            hits++;
            counter++;
            time[j] = counter; // most recently used
            found = 1;
            break;
        }
    }

    // =====
    // PAGE FAULT HANDLING
    // =====

```

```

if(!found) {

    /*
    Page is not in memory → FAULT

    Find **Least Recently Used (LRU)** frame:
    - Frame with smallest `time[]` value
    - Replace that frame with new page
    */

    int lruIndex = 0; // assume frame 0 is LRU initially

    for(int j = 1; j < frames; j++) {
        if(time[j] < time[lruIndex]) {
            lruIndex = j;
        }
    }

    counter++;          // increment global time
    frame[lruIndex] = ref[i]; // load new page
    time[lruIndex] = counter; // update its usage time

    faults++;
}

}

// =====
// FINAL OUTPUT
// =====

printf("\nTotal Page Faults: %d\n", faults);
printf("Total Page Hits: %d\n", hits);

return 0;
}

```